UNIVERSITY OF CAGLIARI
FACULTY OF SCIENCE

Development of Robot Applications

Project Report

Pepper Robot LLM Chatbot

A Conversational AI System for SoftBank Pepper Robot

Submitted by

Name: Balaji Shivakumarappa Bavimane

Matricola: IA/66050

Date: December 30, 2025

Supervised by

Prof. Diego Reforgiato Recupero

Lorenzo Boi

GitHub Repository: https://github.com/balaji1810/Pepper-Robot-Application[1]

[1] Installation and setup instructions are available in the project README at the repository link.

Abstract

This report presents the design and implementation of a conversational AI chatbot for the SoftBank Robotics Pepper humanoid robot. The system enables hands-free voice interaction by integrating Android speech recognition with a LangChain-powered Flask backend utilizing the Mistral large language model. Users speak naturally to Pepper, and the robot responds with contextually appropriate answers through its built-in text-to-speech capabilities. The architecture follows a client-server model where the Android application running on Pepper handles speech capture and synthesis, while the Python backend manages conversational context and LLM inference. This project demonstrates practical integration of modern AI capabilities with social robotics, achieving low-latency conversational interaction suitable for educational and assistive applications. The modular design allows for easy extension and adaptation to different use cases.

Contents

1 Introduction

Human-robot interaction (HRI) has evolved significantly with the advent of large language models (LLMs), enabling more natural and contextually aware conversations between humans and robots. The SoftBank Robotics Pepper robot, designed specifically for social interaction, provides an ideal platform for exploring conversational AI applications. However, integrating modern LLM capabilities with Pepper's existing software ecosystem presents several technical challenges.

This project was undertaken as part of "Development of Robot Applications" course. The primary motivation was to explore the practical integration of LangChain-orchestrated LLM inference with Pepper's native speech capabilities, creating a system where users can engage in natural, hands-free conversations with the robot.

1.1 Motivation

The inbuilt chatbot implementations on Pepper relied on rule-based dialogue systems services with limited contextual understanding plus its very old and not maintained anymore by SoftBank. The emergence of accessible LLM APIs, combined with orchestration frameworks like LangChain, opens opportunities for more sophisticated conversational capabilities. This project investigates how these technologies can be practically deployed in a robotics context.

1.2 Scope and Constraints

The project scope includes:

- Development of a continuous speech recognition system for Pepper
- Implementation of a REST API backend with LangChain and Mistral LLM
- Session-based conversation memory for contextual responses
- Integration with Pepper's QiSDK for text-to-speech synthesis

Out of scope for this initial implementation:

- Multi-modal interaction (gestures, facial expressions)
- On-device LLM inference
- Multi-language support beyond English
- Advanced safety filtering mechanisms

2 Objectives

The project objectives were defined to balance educational learning goals with practical implementation outcomes:

1. **Understand Pepper SDK Integration:** Gain practical experience with the QiSDK for Android, including robot lifecycle management and speech synthesis.
2. **Implement Continuous Speech Recognition:** Develop a hands-free automatic speech recognition system that operates continuously without manual intervention.
3. **Design RESTful API Architecture:** Create a clean, modular Flask API that separates concerns between authentication, request handling, and LLM orchestration.
4. **Integrate LangChain with Mistral:** Implement conversational AI using LangChain's prompt templates and memory management with the Mistral LLM backend.
5. **Manage Conversational Context:** Develop session-based memory storage (in-memory and Redis options) to maintain conversation history across interactions.

3 System Overview

The Pepper Chatbot system follows a distributed client-server architecture where the Android application on Pepper handles user interaction while the Python backend manages AI inference. This separation allows computational intensive LLM processing to occur on more capable hardware while the robot focuses on real-time speech capture and synthesis.

3.1 Information Flow

The conversation flow proceeds through the following stages:

1. User speaks to Pepper's microphone
2. `HandsFreeAsr` component transcribes speech to text using Android's SpeechRecognizer
3. `ChatApiClient` sends the transcribed text via HTTP POST to the Flask API
4. Flask API routes the request to the `ChatAgent` (LangChain chain)
5. `ChatAgent` retrieves conversation history, constructs the prompt, and invokes Mistral LLM
6. LLM response is returned through the API as JSON
7. `ChatBotManager` receives the response and uses QiSDK's `Say` action for speech synthesis
8. Pepper speaks the response; listening automatically resumes

3.2 System Architecture Diagram

Figure 1 illustrates the complete system architecture, showing the interaction between Pepper robot components and the API server.

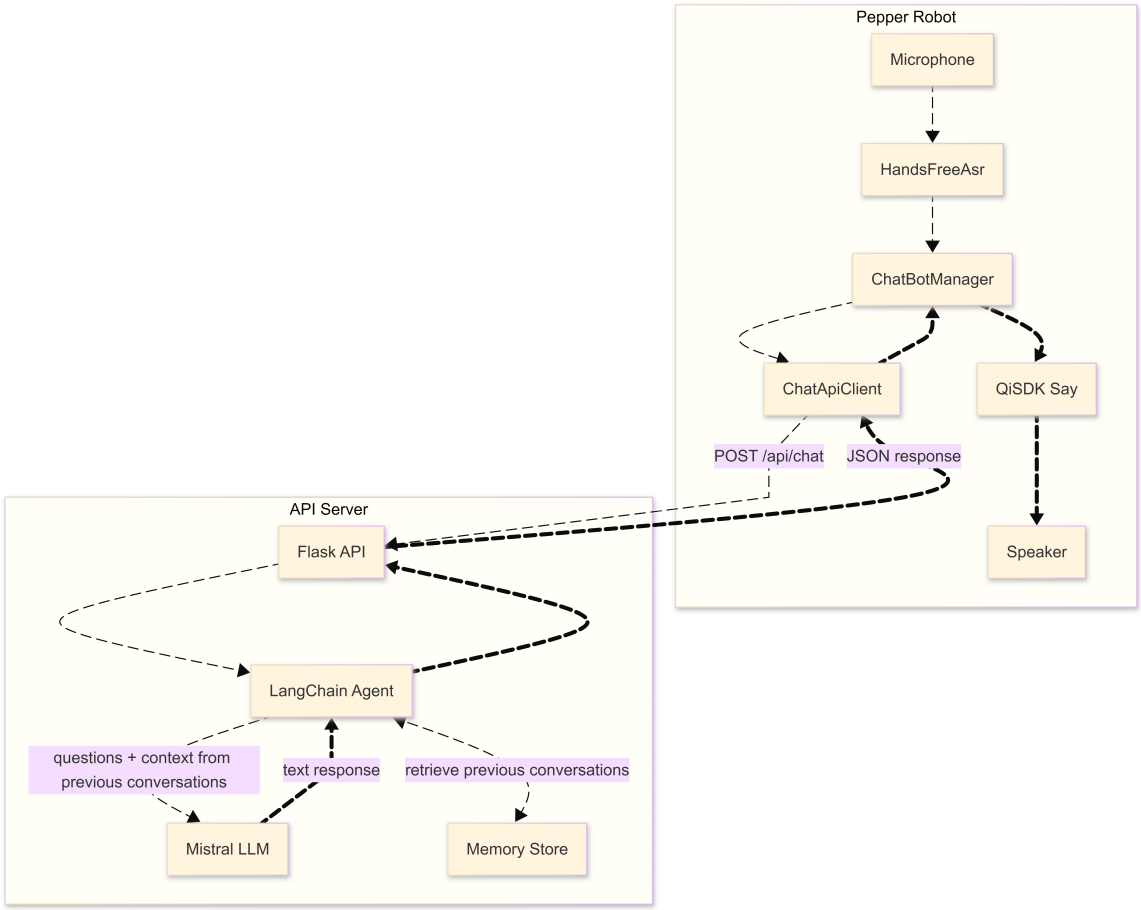

Figure 1: System architecture showing the data flow between Pepper Robot components (right) and the API Server (left).

3.3 Component Summary

Table 1 summarizes the main system components and their responsibilities.

4 Detailed Design

This section provides detailed descriptions of each system component, including design rationale and key implementation aspects.

Table 1: System Component Overview

Component	Description
HandsFreeAsr	Android SpeechRecognizer wrapper providing continuous voice input with automatic restart
ChatBotManager	Orchestrates API calls and Pepper speech synthesis; manages conversation state
ChatApiClient	HTTP client sending JSON messages to the Flask backend
Flask API	REST endpoint wrapping the LangChain agent with Bearer token authentication
ChatAgent	LangChain conversational chain with Mistral LLM and session memory
Memory Store	Session-based storage (in-memory or Redis) for conversation history
QiSDK Say	Pepper's native text-to-speech action for verbal output

4.1 Android Pepper Application

The Android application is structured around four main classes, following separation of concerns principles.

4.1.1 MainActivity

`MainActivity` serves as the central orchestrator, implementing both `RobotLifecycleCallbacks` for QiSDK integration and listener interfaces for ASR and ChatBot events. Key responsibilities include:

- Managing Android activity lifecycle and permissions
- Coordinating robot focus acquisition/loss
- Bridging ASR results to the ChatBot manager
- Updating the conversation display UI

The activity maintains state flags (`isActivityVisible`, `hasRobotFocus`, `hasAudioPermission`) to ensure listening only occurs when all prerequisites are met.

4.1.2 HandsFreeAsr

The `HandsFreeAsr` class encapsulates all speech recognition logic, providing a clean interface to the activity. Key design decisions include:

- **Automatic restart:** After each utterance or physiological error (no speech, no match, timeout), listening automatically restarts with a configurable delay.
- **Error categorization:** Errors are classified as "physiological" (recoverable) or "blocking" (requiring intervention), enabling appropriate handling. See Appendix B.1

Listing 1 shows the configuration of the speech recognition intent.

```
1  private fun createRecognizerIntent(): Intent {
2      return Intent(RecognizerIntent.ACTION_RECOGNIZE_SPEECH).apply {
3          putExtra(RecognizerIntent.EXTRA_LANGUAGE_MODEL, RecognizerIntent.
       LANGUAGE_MODEL_FREE_FORM)
4          putExtra(RecognizerIntent.EXTRA_LANGUAGE, language.toLanguageTag())
5          putExtra(RecognizerIntent.EXTRA_PARTIAL_RESULTS, true)
6          putExtra(RecognizerIntent.EXTRA_MAX_RESULTS, 3)
7          putExtra(RecognizerIntent.EXTRA_SPEECH_INPUT_MINIMUM_LENGTH_MILLIS, 3000L)
8          putExtra(RecognizerIntent.EXTRA_SPEECH_INPUT_COMPLETE_SILENCE_LENGTH_MILLIS, 1500
       L)
9      }
10 }
```

Listing 1: Speech recognition intent configuration in HandsFreeAsr

4.1.3 ChatApiClient

The `ChatApiClient` handles HTTP communication with the Flask backend using Kotlin coroutines for asynchronous operation. The client:

- Constructs JSON request bodies with user text and session ID
- Includes Bearer token authentication headers
- Implements 30-second timeout handling
- Provides sealed class responses (`Success` or `Error`) for type-safe result handling

4.1.4 ChatBotManager

The `ChatBotManager` coordinates the conversation flow, managing three states: `IDLE`, `PROCESSING`, and `SPEAKING`. It:

- Prevents overlapping requests when busy
- Maintains conversation history for UI display
- Invokes QiSDK's `SayBuilder` for speech synthesis
- Requests ASR pause during processing and speaking

Listing 2 shows the speech synthesis implementation.
Appendix B.2 shows different implementations of the states.

```kotlin
private fun speakResponse(text: String) {
    val context = qiContext ?: return
    state = State.SPEAKING
    listener.onSpeakingStarted()

    speakingJob = scope.launch {
        withContext(Dispatchers.IO) {
            val say: Say = SayBuilder.with(context)
                .withText(text)
                .withLocale(Locale(Language.ENGLISH, Region.UNITED_STATES))
                .build()
            say.run()
        }
        state = State.IDLE
        listener.onSpeakingFinished()
        listener.onResumeAsrRequested()
    }
}
```

Listing 2: Speech synthesis using QiSDK in ChatBotManager

4.2 Flask LangChain API

The Python backend follows a modular structure with clear separation between configuration, authentication, routing, and AI logic.

4.2.1 Application Factory Pattern

The `main.py` module uses Flask's application factory pattern, creating the app with configured dependencies:

```python
def create_app() -> Flask:
    config = load_config()

    # Choose memory store based on config
    if config.USE_REDIS and config.REDIS_URL:
        memory = RedisStore(config.REDIS_URL)
    else:
        memory = InMemoryStore()

    agent = ChatAgent(config=config, memory_store=memory)
    app = Flask(__name__)
    app.before_request(require_api_token(config))
    app.register_blueprint(create_api(agent))
```

```
14
15      @app.route("/health")
16      def health():
17          return {"status": "ok"}
18
19      return app
```

Listing 3: Flask application factory in main.py

4.2.2 API Endpoint

The `/api/chat` endpoint handles POST requests with JSON validation:

- Enforces `application/json` content type
- Validates required `text` field (max 2000 characters)
- Auto-generates session IDs if not provided
- Returns structured JSON responses with `ok`, `session_id`, and `reply` fields

4.2.3 Request/Response Format

The API uses a simple JSON schema for communication:

Request:

```
1 {
2     "text": "Hello, how are you?",
3     "session_id": "user123"  // optional
4 }
```

Listing 4: API request JSON schema

Response:

```
1 {
2     "ok": true,
3     "session_id": "user123",
4     "reply": "Hello! I'm doing well, thank you for asking."
5 }
```

Listing 5: API response JSON schema

4.2.4 Authentication

Bearer token authentication is implemented as a Flask `before_request` handler, validating the `Authorization` header against the configured token.

4.3 LangChain Integration

The `ChatAgent` class implements the conversational AI logic using LangChain's composable architecture.

4.3.1 Prompt Engineering

The prompt template includes a system prompt establishing the assistant's persona and constraints:

```
1 SYSTEM_PROMPT = (
2     "You are a helpful assistant for a Pepper robot. "
3     "You must respond politely, concisely (<= 200 words), "
4     "and avoid giving legal/medical advice."
5 )
6
7 PROMPT_TEMPLATE = (
8     "System: " + SYSTEM_PROMPT + "\n\n"
9     "Conversation so far:\n{history}\n\n"
10    "User: {user_input}\nAssistant:"
11 )
```

Listing 6: System prompt and template in chain.py

4.3.2 Chain Construction

The agent uses LangChain Expression Language (LCEL) to compose the processing pipeline:

```
self.llm = ChatMistralAI(
    temperature=config.LLM_TEMPERATURE,
    max_retries=2,
    timeout=30,
)
self.prompt = PromptTemplate(
    input_variables=["history", "user_input"],
    template=PROMPT_TEMPLATE,
)
# Simple chain: prompt -> llm -> parser
self.chain = self.prompt | self.llm | StrOutputParser()
```

Listing 7: LangChain chain construction

See Appendix B.3 to see how to invoke this chain.

4.3.3 Conversation Memory

The memory store maintains session-based conversation history as a list of (user, assistant) tuples. Two implementations are provided:

- `InMemoryStore`: Dictionary-based storage for development/single-instance deployment
- `RedisStore`: Redis list-based storage with TTL (24 hours) for cloud deployment

See Appendix B.4 and Appendix B.5 for detailed implementation.

4.4 Data Flow Diagram

Table 2 summarizes the data transformations at each stage.

Table 2: Data Flow Summary

Stage	Format	Content
Speech Input	Audio	Raw audio from Pepper microphone
ASR Output	String	Transcribed text (e.g., "Hello Pepper")
API Request	JSON	`{text, session_id}`
LLM Prompt	String	Formatted prompt with history
LLM Response	String	Generated response text
API Response	JSON	`{ok, session_id, reply}`
TTS Input	String	Reply text for synthesis
Speech Output	Audio	Pepper's spoken response

5 Implementation Notes

5.1 Threading and Asynchronous Behavior

The Android application uses Kotlin coroutines for asynchronous operations:

- `Dispatchers.IO` for network requests and blocking QiSDK calls
- `Dispatchers.Main` for UI updates
- `SupervisorJob` to prevent coroutine cancellation propagation

5.2 Error Handling Strategy

The system implements error handling:

- **Network errors:** Timeout, connection refused, and other network issues return appropriate HTTP status codes (408, 502, 415, 500...)

- **ASR errors:** Classified into recoverable (auto-restart) and blocking (require user action)
- **LLM errors:** Timeout detection via exception message parsing; automatic retries configured in LangChain

5.3 Configuration Management

Environment-based configuration uses `python-dotenv` for the backend:

Table 3: Configuration Variables

Variable	Description
MISTRAL_API_KEY	API key for Mistral AI (required)
API_AUTH_TOKEN	Bearer token for API authentication (required)
FLASK_HOST	Server bind address (default: 0.0.0.0)
FLASK_PORT	Server port (default: 5000)
USE_REDIS	Enable Redis session storage (0 or 1)
REDIS_URL	Redis connection URL
LLM_TEMPERATURE	LLM creativity parameter (0.0–1.0)

5.4 Security Considerations

The implementation includes basic security measures:

- Bearer token authentication prevents unauthorized API access
- Input length validation (2000 character limit) mitigates prompt injection risks
- API keys are loaded from environment variables, not hardcoded

6 Evaluation and Testing

6.1 Testing Methodology

The system was evaluated through manual testing scenarios on a physical Pepper robot connected to a local network. Testing focused on:

- End-to-end conversation flow
- Error recovery behavior
- Session continuity across multiple exchanges

6.2 Example Dialogues

Table 4 presents representative interaction samples demonstrating system behavior.

Table 4: Example Conversation Exchanges

User Input	Pepper Response
"Hi Pepper, how are you?"	"Hello! I'm doing well, thank you for asking. How can I assist you today?"
"What can you help me with?"	"I can help you with various tasks such as answering questions, providing information, or simply having a conversation. What would you like to know?"
"Tell me a fun fact about robots."	"Here's a fun fact: The word 'robot' comes from the Czech word 'robota,' meaning forced labor. It was first used in a 1920 play by Karel Čapek."

The system successfully maintained conversation context across multiple turns, with the session ID mechanism enabling continuity.

7 Limitations and Risks

7.1 Technical Limitations

- **Network dependency:** Requires stable WiFi connection between Pepper and API server.
- **Single language:** Currently configured for English.
- **No streaming:** Responses are not streamed; full generation must complete before speaking.
- **Memory limitations:** In-memory store loses sessions on server restart.
- **No offline fallback:** System non-functional without API connectivity.

7.2 Known Issues

- ASR may occasionally produce spurious empty results requiring restart.
- Long responses may exceed comfortable listening duration.
- No mechanism to interrupt Pepper while speaking.

7.3 Safety and Privacy Concerns

- **LLM hallucination:** Model may generate incorrect or inappropriate content.
- **Data transmission:** User speech is sent to external Mistral API servers.
- **No content filtering:** Beyond the system prompt, no explicit safety filters are implemented.

8 Discussion and Lessons Learned

8.1 What Worked Well

- **Modular architecture:** Clear separation between components facilitated development and debugging
- **LangChain integration:** The framework simplified LLM orchestration and prompt management
- **QiSDK compatibility:** Pepper's SDK integrated with standard Android patterns after some modifications[2]
- **Conversation history:** Simple tuple-based memory proved effective for maintaining context

8.2 Challenges Encountered

- **QiSDK reliability:** The Pepper SDK for Android provided by SoftBank robotics is relatively old and not maintained properly for latest versions of android studio.
- **Timing coordination:** Preventing ASR from listening while Pepper speaks required careful state management.
- **Response length control:** LLM may sometimes generates longer responses than ideal for spoken interaction.

8.3 Suggestions for Improvement

- Implement response streaming to reduce perceived latency
- Add configurable wake word detection like "Ok Pepper" or "Hey Pepper" to automatically start listening
- Integrate gesture and expression alongside speech
- Deploy safety filters (toxicity detection, topic restrictions)

9 Future Work

- **Safety and Content Filtering:** Implement moderation API calls or local classifiers to filter inappropriate LLM outputs before speech synthesis.
- **Response Streaming:** Use Server-Sent Events or WebSockets to stream partial responses, enabling Pepper to begin speaking before full generation completes.

[2]The "QiSDK Plugin" for Android Studio was published in 2017, but as of 2025 it is no longer compatible with Android Studio. I used this guide to use latest version of Android Studio with Pepper, without the need of the plugin.

- **On-Device Inference:** Explore smaller LLMs (e.g., Phi, Gemma) that could run on a companion device for offline operation.
- **Multi-Modal Interaction:** Integrate Pepper's gesture capabilities to complement verbal responses with appropriate body language.
- **Multi-Language Support:** Extend ASR configuration to support multiple languages with automatic detection.
- **User Personalization:** Store user preferences and adapt responses based on interaction history.

10 Conclusions

This project successfully demonstrated the integration of modern large language model capabilities with the SoftBank Pepper robot platform. The resulting system enables natural, hands-free conversational interaction where users speak to Pepper and receive contextually appropriate verbal responses.

Key achievements include:

- A functional end-to-end conversational system from speech input to robot speech output
- Clean, modular architecture separating Android client and Python backend concerns
- Session-based conversation memory enabling multi-turn dialogue
- Comprehensive error handling for reliable operation

The project provided valuable learning experiences in robot application development, Android SDK integration, REST API design, and LLM orchestration with LangChain. While limitations exist, particularly regarding safety filtering and offline operation, the system provides a solid foundation for future enhancements and demonstrates the feasibility of deploying LLM-powered conversational AI on social robots.

References

[1] Mistral AI. Mistral API Documentation: https://docs.mistral.ai/.

[2] Balaji S. Bavimane.
GitHub Repository: https://github.com/balaji1810/Pepper-Robot-Application.

[3] Google. Android Developers Documentation: https://developer.android.com.

[4] LangChain. LangChain Python Documentation:
https://docs.langchain.com/oss/python/langchain/overview.

[5] Pallets. Flask Web Framework: https://flask.palletsprojects.com/.

[6] SoftBank Robotics. QiSDK Documentation:
https://qisdk.softbankrobotics.com/sdk/doc/pepper-sdk/index.html.

A Libraries and Versions

A.1 Python Backend:

- flask 3.1.2
- python-dotenv 1.2.1
- redis 7.1.0
- langchain 1.2.0
- langchain-mistralai 1.1.1

A.2 Android Application:

- Kotlin 2.0.21
- Android Gradle Plugin 8.13
- QiSDK 1.7.5
- AndroidX Core KTX 1.10.1
- AndroidX Compose BOM 2024.09.00
- AndroidX Activity Compose 1.8.0

B Selected Code Excerpts

B.1 ASR Error Handling

Error categorization for appropriate recovery behavior:

```
sealed class AsrError(val message: String) {
    // Physiological errors - can automatically restart
    class NoSpeech : AsrError("No speech detected")
    class NoMatch : AsrError("Speech not recognized")
    class Timeout : AsrError("Speech timeout")

    // Blocking errors - require user intervention
    class PermissionDenied : AsrError("Microphone permission denied")
    class AudioError : AsrError("Audio recording error")
    class NetworkError : AsrError("Network error")
    class NotAvailable : AsrError("Speech recognition not available")

    val isPhysiological: Boolean
        get() = this is NoSpeech || this is NoMatch || this is Timeout
}
```

Listing 8: ASR error handling from HandsFreeAsr.kt

B.2 States of the application

Different states of `ChatBotManager`. When $state \neq IDLE$ any new message from the user will be ignored.

```
enum class State {
    IDLE,           // Waiting for user input
    PROCESSING,     // Sending to API and waiting for response
    SPEAKING        // Pepper is speaking the response
}
```

Listing 9: enum State from ChatBotManager.kt

Different states of speech recognizer. When $state = PROCESSING$ speech recognition will be paused temporarily.

```
// Represents the internal state of the ASR.
enum class State {
    IDLE,           // Not listening, microphone inactive
    LISTENING,      // Microphone active, waiting for speech
    SPEECH_ACTIVE,  // User is currently speaking
    PROCESSING      // Processing the utterance (after speech ends)
}
```

Listing 10: enum State from HandsFreeAsr.kt

Each state in the `ChatState` enum represents a phase in the conversation flow:

- `INITIALIZING` – App is starting up and setting up components
- `IDLE` – System is ready but not actively listening or processing
- `LISTENING` – ASR is active and waiting for the user to speak
- `USER_SPEAKING` – User has started speaking and voice is being captured
- `PROCESSING` – User's speech has ended; sending to API and awaiting response
- `BOT_SPEAKING` – Pepper robot is speaking the bot's response aloud
- `ERROR` – Something went wrong (permission denied, ASR unavailable, API error, etc.)

```
// Represents the overall chat state for UI updates.
enum class ChatState {
    INITIALIZING,
    IDLE,
    LISTENING,
    USER_SPEAKING,
    PROCESSING,
    BOT_SPEAKING,
    ERROR
}
```

Listing 11: enum ChatState from MainActivity.kt

B.3 ChatAgent Reply Method

The core method that processes user messages and generates responses:

```
def reply(self, session_id: str, text: str) -> str:
    history_pairs = self.memory_store.get(session_id)
    history_str = format_history(history_pairs)

    try:
        response: str = self.chain.invoke({
            "history": history_str,
            "user_input": text,
        })
    except Exception as e:
        msg = str(e).lower()
        if "timeout" in msg or "timed out" in msg:
            raise TimeoutError("LLM timeout")
        raise

    # Update memory
    try:
        self.memory_store.append(session_id, text, response)
    except Exception:
        logger.exception("Failed to append to memory store")

    return response
```

Listing 12: ChatAgent.reply() method from chain.py

B.4 RedisStore Implementation

Cloud session storage using Redis lists:

```
ConversationPair = tuple[str, str]

class RedisStore():
    def __init__(self, redis_url: str, ttl_seconds: int = 86400) -> None:
        self._client = redis.Redis.from_url(redis_url, decode_responses=True)
        self._ttl = ttl_seconds
        self._sep = "\u241E"  # Separator for user/assistant text

    def _key(self, session_id: str) -> str:
        return f"pepper:chat:{session_id}"

    def get(self, session_id: str) -> list[ConversationPair]:
```

```
        key = self._key(session_id)
        items = self._client.lrange(key, 0, -1)
        result = []
        for raw in items:
            user, assistant = raw.split(self._sep, 1)
            result.append((user, assistant))
        if result:
            self._client.expire(key, self._ttl)  # Extend TTL
        return result

    def append(self, session_id: str, user_text: str,
               assistant_text: str) -> None:
        key = self._key(session_id)
        payload = f"{user_text}{self._sep}{assistant_text}"
        pipeline = self._client.pipeline()
        pipeline.rpush(key, payload)
        pipeline.expire(key, self._ttl)
        pipeline.execute()
```

Listing 13: RedisStore class from memory_store.py

B.5 InMemoryStore Implementation

In memory session storage using python's list and tuple

```

class InMemoryStore():

    def __init__(self) -> None:
        self._store: dict[str, list[ConversationPair]] = {}

    def get(self, session_id: str) -> list[ConversationPair]:
        return list(self._store.get(session_id, []))

    def append(self, session_id: str, user_text: str, assistant_text: str) -> None:
        self._store.setdefault(session_id, []).append((user_text, assistant_text))
```

Listing 14: InMemoryStore class from memory_store.py

C Installation Note

Complete installation and setup instructions, including Python environment configuration, Android Studio setup, and Pepper robot deployment procedures, are available in the project README at:

https://github.com/balaji1810/Pepper-Robot-Application